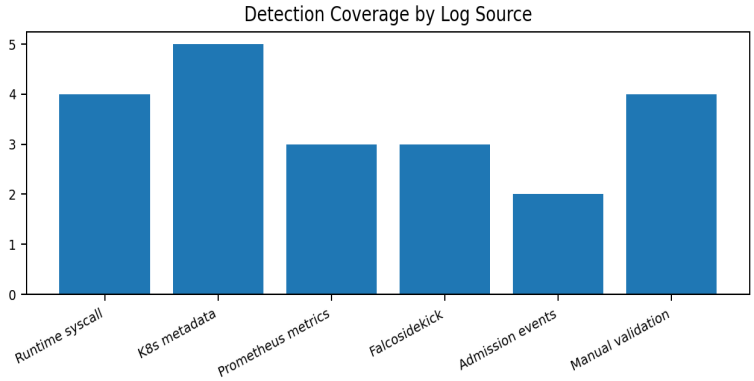
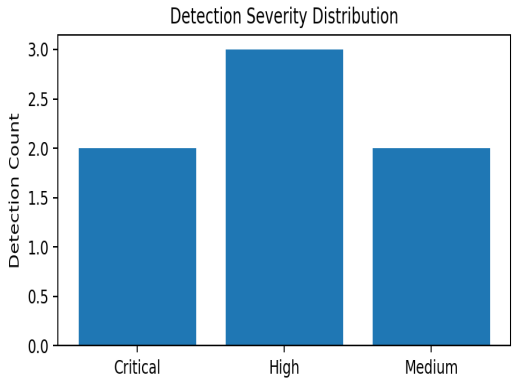
Enterprise DevSecOps Runtime Detection Report

Falco + Falcosidekick + Prometheus + Grafana Detection Engineering Evidence

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Focus Area	Runtime detection engineering, Kubernetes attack detection, observability, triage and response
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, Falco, Falcosidekick, Prometheus, Grafana
Document Type	Professional detection report and response playbook

This report documents enterprise-style runtime detections built from the project phase that deployed Falco, Falcosidekick, Prometheus, and Grafana. Each detection includes a name, MITRE ATT&CK; mapping, log source, trigger logic, false positives, severity, triage steps, response steps, and screenshot evidence from the private lab.





Report outcome: The lab implements both preventive controls and detective controls. Preventive controls include Pod Security, RBAC least privilege, and NetworkPolicy. Detective controls include Falco runtime rules, Falcosidekick forwarding, Prometheus metrics, Prometheus alert rules, and Grafana dashboards.

1. Executive Summary

The detection engineering phase transforms raw Kubernetes runtime behavior into actionable security detections. Falco provides runtime telemetry from Linux syscall activity and Kubernetes/container metadata. Falcosidekick forwards Falco events into the monitoring ecosystem. Prometheus stores time-series detection metrics, and Grafana visualizes runtime detections, event rates, rule priorities, and detection health.

The lab validates detections against controlled attack simulations: privileged pod creation, hostPath abuse, chroot access to the node filesystem, sensitive file reads, interactive shell execution, RBAC over-privilege, NetworkPolicy enforcement, and Falco alert/metrics visibility. The approach is intentionally enterprise-oriented: every detection is mapped to a threat behavior, has defined triage and response steps, and includes screenshot evidence.

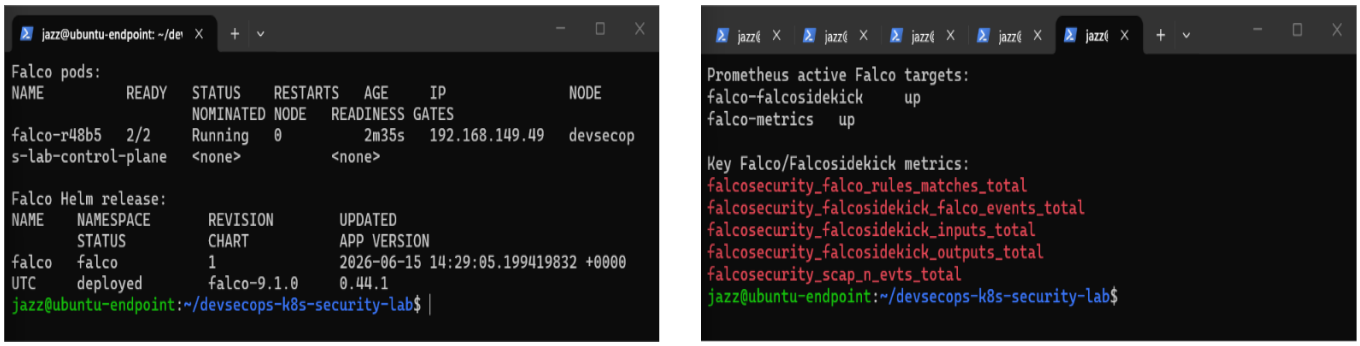
Detection ID	Detection Name	Severity	Primary Evidence
DET-K8S-001	Privileged Container Started with Host Filesystem Access Pattern	Critical	Privileged attacker pod running
DET-K8S-002	Sensitive File Access from Container	High	Falco sensitive file alert evidence
DET-K8S-003	Interactive Shell or Command Execution Inside Container	High	Falco live log watch for runtime events
DET-K8S-004	Overprivileged Kubernetes ServiceAccount and Cluster-Admin Binding	High	Overprivileged service account can get secrets before fix
DET-K8S-005	Falco Runtime Detection Metrics Alert	Medium	Prometheus scraping Falco targets
DET-K8S-006	Falco Telemetry Drop or Detection Pipeline Health Degradation	High	Falco and Falcosidekick running
DET-K8S-007	Untrusted Pod Network Access Blocked by NetworkPolicy	Medium	NetworkPolicy blocks untrusted pod access

Severity rationale: Critical findings represent likely host escape or host-level compromise patterns. High findings represent credential exposure, excessive Kubernetes privileges, or telemetry gaps affecting detection reliability. Medium findings represent network segmentation validation or general visibility signals that require correlation.

2. Detection Architecture and Log Source Model

The detection pipeline is designed as an evidence chain. Falco observes runtime events, Falcosidekick forwards events and exposes a security-friendly interface, Prometheus scrapes metrics and evaluates alert expressions, and Grafana presents the operational view. The same workflow also supports post-incident evidence capture because each detection can be tied back to namespace, pod, container, image, command line, rule name, priority, and timestamp.

Layer	Component	Security Function	Evidence Produced
Runtime sensor	Falco	Detects suspicious syscall and container activity using rule conditions and priorities.	Falco logs, JSON fields, rule name, priority, process/file/container metadata
Event forwarding	Falcosidekick	Forwards Falco alerts to external systems and exposes operational views.	Forwarded event stream, UI evidence, metrics integration
Metrics store	Prometheus	Scrapes Falco/Falcosidekick metrics and evaluates alerting rules.	falcosecurity_* metrics, active targets, PrometheusRule status
Dashboard	Grafana	Visualizes runtime detections and detection health.	Falco security dashboard, top rules, event priority panels
Kubernetes context	Kind + Kubernetes metadata	Provides workload identity: namespace, pod, service account, labels, node.	kubectl describe/get output, events, labels, service account mapping



Evidence: Falco/Falcosidekick deployment and Prometheus scrape target validation.

3. Detection Rule Design Principles

The detection rules are written to support SOC-style triage rather than simply proving a lab command executed. Each rule identifies who performed the action, what happened, which asset was affected, why it matters, and how the response should proceed. Detection logic is treated as an engineering artifact with defined scope, trigger, expected false positives, severity, and validation criteria.

- Prefer high-context detections: include namespace, pod, container, image, command line, file path, rule name, and priority where possible.
- Separate true attack indicators from operational noise through scoped allowlists rather than disabling rules globally.
- Create Prometheus alerts for both security activity and detection pipeline health.
- Tie each alert to triage steps, response steps, screenshots, and retest commands.
- Use MITRE ATT&CK mapping as a communication layer, not as a replacement for environment-specific investigation.

4. DET-K8S-001: Privileged Container Started with Host Filesystem Access Pattern

Field	Detection Detail
Detection name	Privileged Container Started with Host Filesystem Access Pattern
Severity	Critical
MITRE ATT&CK; mapping	T1611 Escape to Host; T1610 Deploy Container; T1059 Command and Scripting Interpreter
Log source	Falco syscall events from Kubernetes nodes; container runtime metadata; Kubernetes pod metadata; Falcosidekick forwarded alerts; Prometheus rule-match metrics.
Trigger condition	A container is started with elevated runtime privileges, hostPath access to the node filesystem, and subsequent shell or chroot activity referencing /host. In the lab this was triggered by attacker-privileged with securityContext.privileged=true and hostPath path=/ mounted at /host.
PromQL / detection expression	sum by (rule_name, priority, k8s_ns_name, k8s_pod_name) (increase(falcosecurity_falco_rules_matches_total{rule_name=~".*Privileged.*|.*sensitive.*"}[5m])) > 0

False Positives to Validate

- Privileged security tooling such as node agents, CNI components, storage drivers, or monitoring DaemonSets.
- Break-glass maintenance containers approved by platform operations.
- Vendor-provided privileged workloads that are explicitly documented and namespace-restricted.

Triage Steps

1. Identify namespace, pod, container image, service account, node, command line, and parent process from Falco output_fields.

2. Check whether the pod spec includes privileged=true, hostPID, hostNetwork, hostPath, or mounted docker/containerd sockets.

3. Confirm whether the image digest and workload owner are approved in the deployment baseline.

4. Review recent Kubernetes events and GitOps drift to determine whether the workload came from Git or direct kubectl activity.

5. Check for subsequent sensitive file reads, shell execution, network connections, or RBAC operations from the same pod.

Response Steps

1. Isolate or delete the pod if unauthorized: kubectl delete pod <pod> -n <namespace>.

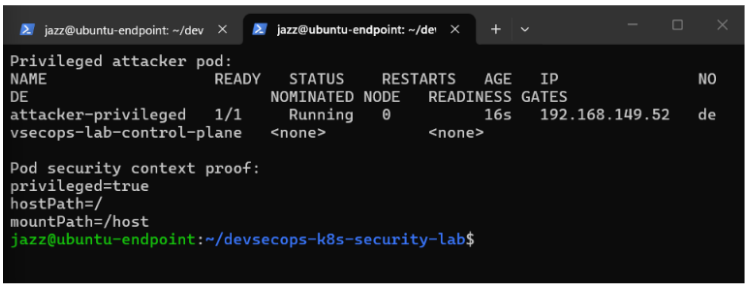
2. Capture evidence first when possible: pod YAML, Falco alert JSON, container image digest, node name, and involved service account.

3. Apply namespace Pod Security restricted enforcement and Gatekeeper/Kyverno policy to block privileged containers and hostPath mounts.

4. Rotate any credentials that may have been accessible through hostPath or mounted service account tokens.

5. Create a remediation ticket requiring owner approval before privileged workload exceptions are accepted.

Evidence Screenshot



Privileged attacker pod running (134-privileged-attacker-pod-running.png)

Additional evidence available: 135-container-escape-chroot-proof.png - Container escape proof through chroot /host; 136-falco-detected-privileged-pod-activity.png - Falco detection for privileged pod activity

5. DET-K8S-002: Sensitive File Access from Container

Field	Detection Detail
Detection name	Sensitive File Access from Container
Severity	High
MITRE ATT&CK; mapping	T1552 Unsecured Credentials; T1552.001 Credentials In Files; T1552.007 Container API where service account or container credentials are involved.
Log source	Falco syscall events, Falco default rule output, container process context, file path metadata, Falcosidekick event stream, and Prometheus Falco rule-match metric.
Trigger condition	A process inside a container reads sensitive files such as /etc/shadow, /host/etc/shadow, service account tokens, kubeconfig files, private keys, shell history, or cloud credential files. In the lab, the event was safely triggered by reading /etc/shadow or /host/etc/shadow without exposing the secret content.
PromQL / detection expression	sum by (rule_name, priority, source) (increase(falcosecurity_falco_rules_matches_total{rule_name=~".*sensitive.*|.*shadow.*|.*credential.*"}[10m])) > 0

False Positives to Validate

- Security scanners that inspect images or mounted filesystem content under controlled conditions.
- Backup, inventory, or compliance agents with approved read-only access to system files.
- Base image initialization scripts that briefly inspect local user/group files during startup.

Triage Steps

1. Inspect the exact file path, process name, command line, container image, namespace, and pod labels from the alert.
2. Determine whether the file path is from the container filesystem or a mounted host filesystem under /host.
3. Validate whether the workload normally requires access to credential material; most application pods should not.
4. Search for adjacent alerts from the same container: shell spawn, privilege escalation, network connection, or hostPath access.
5. Check whether service account token automount was enabled and whether credentials could be used against the Kubernetes API.

Response Steps

1. Preserve the alert details and container metadata, then isolate the pod if unauthorized.
2. Rotate any secrets or service account tokens that may have been read or exposed.
3. Patch deployment manifests to disable service account token automount unless required.
4. Apply read-only root filesystem, non-root user, and restricted Pod Security profiles.
5. Add detection exceptions only for documented security tools with namespace and image digest scoping.

Evidence Screenshot

Falco sensitive file alert evidence (133-falco-basic-alert-sensitive-file.png)

Additional evidence available: 155-falco-events-generated.png - Falco events generated for Grafana/Prometheus validation

6. DET-K8S-003: Interactive Shell or Command Execution Inside Container

Field	Detection Detail
Detection name	Interactive Shell or Command Execution Inside Container
Severity	High
MITRE ATT&CK; mapping	T1059 Command and Scripting Interpreter; T1082 System Information Discovery when commands gather host, user, or environment information.
Log source	Falco process execution events, command-line fields, container ID, Kubernetes metadata, live Falco logs, and Falcosidekick/Prometheus metrics.
Trigger condition	A shell such as sh, bash, ash, or zsh is executed inside a running container outside the approved baseline. In the lab, shell activity occurred during controlled kubectl exec and chroot validation of the privileged pod.
PromQL / detection expression	sum by (rule_name, priority, k8s_ns_name, k8s_pod_name) (increase(falcosecurity_falco_rules_matches_total{rule_name=~".*shell.*|.*Terminal.*"}[5m])) > 0

False Positives to Validate

- Developer troubleshooting using kubectl exec in a non-production namespace.
- Init containers or entrypoints that legitimately invoke shell scripts.
- Approved maintenance workflows performed by platform engineers with ticketed change approval.

Triage Steps

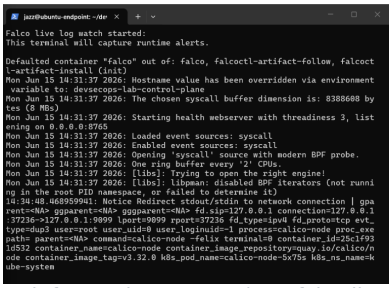
1. Review the executed command line and whether the process was launched by kubectl exec, container entrypoint, or application process.
2. Map the user, service account, namespace, pod owner, and image digest to the approved operations baseline.
3. Determine whether commands were discovery-only or attempted credential access, network movement, or host filesystem access.
4. Correlate with Kubernetes audit logs, CI/CD events, and ArgoCD sync state to identify whether live drift occurred.
5. Check for repeated shell sessions across multiple pods or namespaces, which may indicate lateral movement.

Response Steps

1. Terminate unauthorized interactive sessions and isolate the affected pod or namespace.
2. Review Kubernetes RBAC permissions allowing exec/attach/port-forward actions.
3. Restrict kubectl exec in production through RBAC and operational approvals.
4. Add namespace labels and pod labels to reduce false positives for approved maintenance windows.

5. If suspicious, collect pod logs, events, Falco JSON, image digest, and service account permissions for incident review.

Evidence Screenshot



Falco live log watch for runtime events (131-falco-live-log-watch.png)

Additional evidence available: 135-container-escape-chroot-proof.png - Command execution and chroot validation evidence

7. DET-K8S-004: Overprivileged Kubernetes ServiceAccount and Cluster-Admin Binding

Field	Detection Detail
Detection name	Overprivileged Kubernetes ServiceAccount and Cluster-Admin Binding
Severity	High
MITRE ATT&CK; mapping	T1552.007 Container API for Kubernetes API credential abuse; related control-plane privilege escalation pattern through overprivileged service accounts.
Log source	Kubernetes audit events where available; kubectl auth can-i validation output; clusterrolebinding inventory; Gatekeeper/admission logs; CI/CD evidence and evidence screenshots.
Trigger condition	A non-platform service account is granted cluster-admin or broad get/list/watch/create permissions across all namespaces. In the lab, overprivileged-sa was intentionally bound to cluster-admin and could get secrets cluster-wide before remediation.
PromQL / detection expression	Not directly Prometheus-only. Use Kubernetes audit detection or periodic RBAC inventory, then export policy violations as metrics where available.

False Positives to Validate

- Cluster lifecycle controllers, GitOps controllers, or security controllers that require cluster-wide access and are explicitly approved.
- Temporary break-glass administrative service accounts created during incident recovery and removed immediately afterward.
- Lab-only demonstration service accounts intentionally created to prove RBAC risk.

Triage Steps

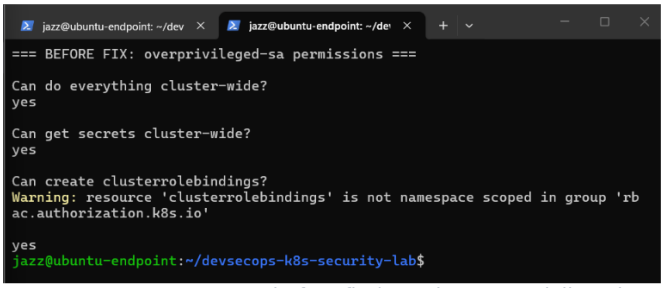
1. Identify the ClusterRoleBinding, subject kind, subject namespace, roleRef, creation time, and actor that created the binding.
2. Run kubectl auth can-i tests for secrets, pods/exec, clusterrolebindings, and nodes using the affected service account identity.
3. Check whether the service account token is mounted in any running pod and whether automountServiceAccountToken is enabled.
4. Correlate with recently created pods, suspicious kubectl exec sessions, or secret access attempts.
5. Review Git history and ArgoCD state to determine whether the binding came from Git or live-cluster drift.

Response Steps

1. Delete unauthorized cluster-admin binding immediately: kubectl delete clusterrolebinding <binding>.

2. Replace with namespace-scoped Role and RoleBinding that grants only required verbs/resources.
3. Rotate service account tokens if the account was mounted into any pod with suspicious activity.
4. Add CI/CD and admission policies to block cluster-admin bindings for non-approved subjects.
5. Document approved cluster-admin exceptions with owner, reason, expiry, and compensating controls.

Evidence Screenshot



Overprivileged service account can get secrets before fix (140-rbac-overprivileged-sa-can-get-secrets.png)

Additional evidence available: 143-rbac-fixed-cannot-get-secrets.png - RBAC fixed: service account cannot get secrets after remediation

8. DET-K8S-005: Falco Runtime Detection Metrics Alert

Field	Detection Detail
Detection name	Falco Runtime Detection Metrics Alert
Severity	Medium
MITRE ATT&CK; mapping	Technique-specific mapping inherited from the triggered Falco rule, commonly T1611, T1059, or T1552 for this lab.
Log source	Prometheus metrics from Falco and/or Falcosidekick, ServiceMonitor discovery, PrometheusRule alert evaluations, and Grafana panels.
Trigger condition	Any Falco rule match is observed in a defined time window, for example increase(falcosecurity_falco_rules_matches_total[5m]) > 0. This detection converts raw runtime events into monitored security signals for dashboarding and alerting.
PromQL / detection expression	sum by (rule_name, priority, source) (increase(falcosecurity_falco_rules_matches_total[5m])) > 0

False Positives to Validate

- Expected lab simulations, red-team exercises, or training runs.
- Approved administrative activity in a maintenance window.
- High-volume but low-risk default rules that require environment-specific tuning.

Triage Steps

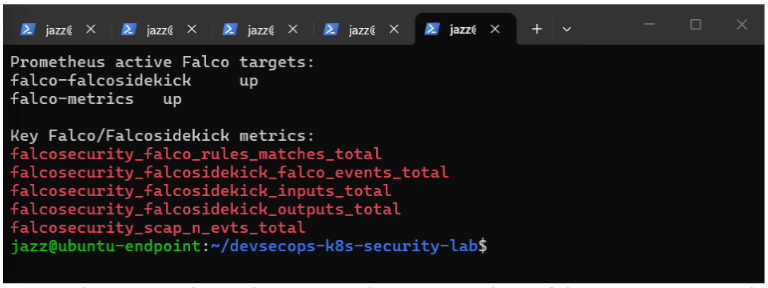
1. Open Grafana Falco Runtime Security Dashboard and identify the rule name, priority, namespace, pod, and time range.
2. Query Prometheus for the rule-name label and compare current rate with the historical baseline.
3. Pull Falco logs for the same time window and capture full output_fields for the event.
4. Check whether the alert came from a lab simulation or an unexpected namespace/workload.
5. Escalate if the event involves privileged containers, host filesystem access, secrets, or repeated shell execution.

Response Steps

1. Acknowledge and classify the alert as benign lab test, true positive, or tuning candidate.
2. If true positive, isolate pod/namespace and collect Falco, Kubernetes, and container runtime evidence.
3. Create or update a PrometheusRule with appropriate severity and labels.

4. If noisy, tune with namespace/image allowlists rather than disabling the rule globally.
5. Update the detection report with false-positive notes and response outcome.

Evidence Screenshot



Prometheus scraping Falco targets (154-prometheus-falco-targets-up.png)

Additional evidence available: 156-prometheus-falco-rule-matches-query.png - Prometheus Falco rule matches query; 158-custom-falco-security-dashboard-imported.png - Custom Grafana Falco security dashboard

9. DET-K8S-006: Falco Telemetry Drop or Detection Pipeline Health Degradation

Field	Detection Detail
Detection name	Falco Telemetry Drop or Detection Pipeline Health Degradation
Severity	High
MITRE ATT&CK; mapping	Defensive telemetry health control; not a direct adversary technique. If deliberate, treat as potential defense evasion investigation.
Log source	Falco metrics exposed in Prometheus format; Prometheus alerting rules; Grafana operational panels; Kubernetes pod health for Falco and Falcosidekick.
Trigger condition	Falco kernel event drops or output queue drops are greater than zero over a defined window, for example increase(falcosecurity_scap_n_drops_total[5m]) > 0 or increase(falcosecurity_falco_outputs_queue_num_drops_total[5m]) > 0.
PromQL / detection expression	sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0 or sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0

False Positives to Validate

- Short resource pressure during lab cluster startup or upgrade.
- Prometheus scrape gaps during pod restarts.
- Temporary increase in syscall volume from controlled test workloads.

Triage Steps

1. Check Falco pod CPU and memory usage, restart count, and node pressure status.
2. Review Prometheus target health and last scrape error for Falco/Falcosidekick targets.
3. Inspect Falco logs for dropped event warnings or output plugin backpressure.
4. Correlate with a burst of runtime alerts or high-volume test activity.
5. Determine whether detection fidelity was affected during a security event.

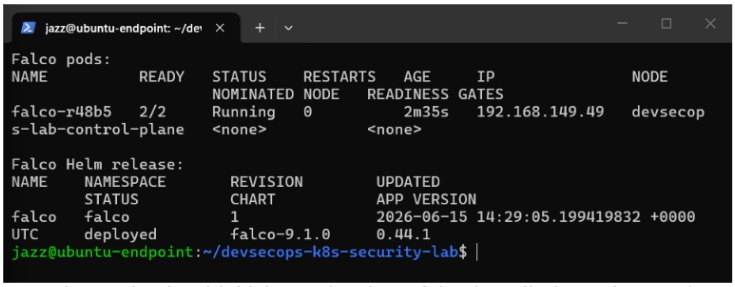
Response Steps

1. Scale or resource-tune Falco/Falcosidekick where possible.
2. Reduce noisy rules only after confirming they are not security-critical.
3. Increase retention and scrape stability for the monitoring stack.

4. If drops occurred during a suspected attack, treat the period as degraded visibility and collect supporting Kubernetes/runtime logs.

5. Create an operational ticket to tune capacity and alert thresholds.

Evidence Screenshot



Falco and Falcosidekick running (130-falco-installed-running.png)

Additional evidence available: 160-falco-prometheus-alert-rules.png - Prometheus security alert rules for Falco

10. DET-K8S-007: Untrusted Pod Network Access Blocked by NetworkPolicy

Field	Detection Detail
Detection name	Untrusted Pod Network Access Blocked by NetworkPolicy
Severity	Medium
MITRE ATT&CK; mapping	T1610 Deploy Container when a pod is created to bypass normal controls; network movement investigation depends on destination and behavior.
Log source	Kubernetes NetworkPolicy validation output, Calico-enforced traffic behavior, application service reachability tests, and optional CNI flow logs if enabled.
Trigger condition	A pod without approved labels attempts to reach demo-app service and times out or is blocked by the default-deny and allow-list NetworkPolicy model. In the lab, blocked-curl could not access demo-app-svc while allowed pod traffic succeeded.
PromQL / detection expression	NetworkPolicy reachability validation is command/test based in this lab. Optional production enhancement: export CNI flow logs to Prometheus/Loki for alerting.

False Positives to Validate

- New application pods missing the required access label due to deployment misconfiguration.
- Health-check or test pods not included in the network policy allow-list.
- Namespace migration where policies were applied before labels or ServiceMonitors were updated.

Triage Steps

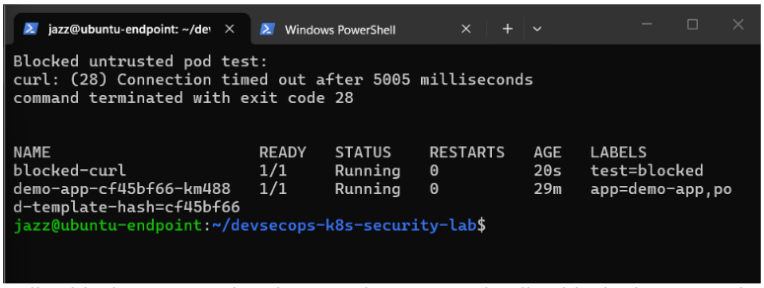
1. Identify source namespace, pod labels, destination service, port, and NetworkPolicy selectors.
2. Confirm whether the source pod should have access based on application architecture.
3. Compare blocked source labels against allow-demo-app-ingress policy and namespaceSelector rules.
4. Check whether DNS egress is allowed and whether the failure is network policy vs service discovery.
5. Review recent deployment changes that may have removed required labels.

Response Steps

1. If the pod is unauthorized, leave policy enforcement in place and investigate workload origin.
2. If the pod is legitimate, add the correct label or update policy with owner approval.
3. Avoid broad CIDR or namespace-wide allows unless justified by architecture.

4. Document the exception with owner, service, source, destination, port, and expiry.
5. Retest blocked and allowed paths after policy changes.

Evidence Screenshot



NetworkPolicy blocks untrusted pod access (084-networkpolicy-blocked-untrusted-pod.png)

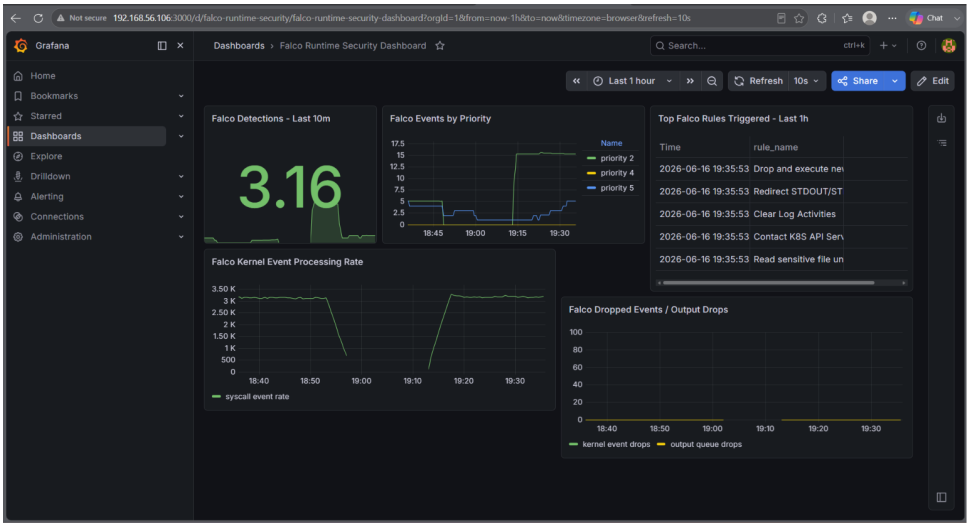
11. Prometheus Alerting and Dashboard Operationalization

The project converts Falco detections into measurable security signals. Prometheus evaluates rule-match increases, event drops, and output queue health. Grafana provides analyst visibility into top rule names, event priority, detection volume, and detection pipeline reliability.

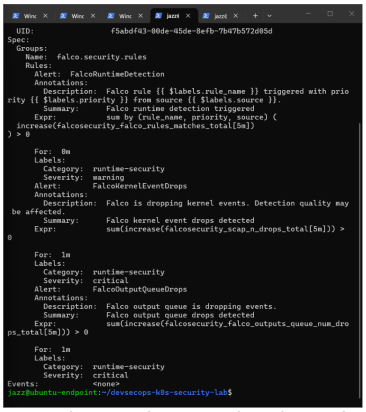
```
- alert: FalcoRuntimeDetection
  expr: sum by (rule_name, priority, source) (
          increase(falcosecurity_falco_rules_matches_total[5m])
        ) > 0
  labels:
    severity: warning
    category: runtime-security

- alert: FalcoKernelEventDrops
  expr: sum(increase(falcosecurity_scap_n_drops_total[5m])) > 0
  labels:
    severity: critical
    category: runtime-security

- alert: FalcoOutputQueueDrops
  expr: sum(increase(falcosecurity_falco_outputs_queue_num_drops_total[5m])) > 0
  labels:
    severity: critical
    category: runtime-security
```



Grafana custom Falco Runtime Security Dashboard



Prometheus Falco security alert rules

12. Analyst Triage Workflow

The following workflow is intended for a junior analyst or security engineer reviewing a Falco alert in the private lab or adapting the same approach to an enterprise Kubernetes environment.

Step	Action	Command or Evidence	Decision Point
1	Confirm alert freshness	Grafana time range, Prometheus query, Falco logs --since=10m	Is this current or historical?
2	Identify workload	namespace, pod, container, image, node, service account	Is workload approved?
3	Classify behavior	rule_name, priority, command line, file path, parent process	Expected maintenance or suspicious action?
4	Check blast radius	RBAC can-i, mounted volumes, service account token, NetworkPolicy	Can the pod access secrets, host, or other services?
5	Contain if needed	kubectl delete pod, scale deployment, apply network isolation	Does isolation break production?
6	Remediate root cause	Pod Security, RBAC least privilege, read-only FS, Gatekeeper, NetworkPolicy	Is control now preventive?
7	Retest and close	Replay safe test, confirm Falco/Prometheus/Grafana evidence	Detection and fix both validated?

13. Evidence Index

Evidence File	Purpose
134-privileged-attacker-pod-running.png	Privileged attacker pod running
135-container-escape-chroot-proof.png	Container escape proof through chroot /host
136-falco-detected-privileged-pod-activity.png	Falco detection for privileged pod activity
133-falco-basic-alert-sensitive-file.png	Falco sensitive file alert evidence
155-falco-events-generated.png	Falco events generated for Grafana/Prometheus validation
131-falco-live-log-watch.png	Falco live log watch for runtime events
140-rbac-overprivileged-sa-can-get-secrets.png	Overprivileged service account can get secrets before fix
143-rbac-fixed-cannot-get-secrets.png	RBAC fixed: service account cannot get secrets after remediation
154-prometheus-falco-targets-up.png	Prometheus scraping Falco targets
156-prometheus-falco-rule-matches-query.png	Prometheus Falco rule matches query
158-custom-falco-security-dashboard-imported.png	Custom Grafana Falco security dashboard
130-falco-installed-running.png	Falco and Falcosidekick running
160-falco-prometheus-alert-rules.png	Prometheus security alert rules for Falco
084-networkpolicy-blocked-untrusted-pod.png	NetworkPolicy blocks untrusted pod access
detection_coverage_sources.png	Detection coverage by log source chart
detection_severity_distribution.png	Severity distribution chart

14. References and Technical Basis

Reference	Use in this report
MITRE ATT&CK; T1611 - Escape to Host	Mapped container escape and privileged hostPath activity to container host escape behavior.
MITRE ATT&CK; T1610 - Deploy Container	Mapped risky container deployment behavior and unauthorized pod creation patterns.
MITRE ATT&CK; T1059 - Command and Scripting Interpreter	Mapped shell and command execution inside containers.
MITRE ATT&CK; T1082 - System Information Discovery	Mapped host/user/environment discovery commands observed during validation.
MITRE ATT&CK; T1552, T1552.001, T1552.007	Mapped sensitive file/credential access and Kubernetes API credential risks.
Falco rule documentation	Used for rule fields, priority interpretation, conditions, output fields, and runtime rule design.
Falco outputs and Falcosidekick documentation	Used for event forwarding and integration with external monitoring systems.
Falco metrics documentation	Used for Prometheus metric names and runtime detection dashboards.
Prometheus alerting rules documentation	Used for alert expression, label, annotation, and firing-condition model.

Note: This report is based on a private lab implementation. In production, attach Kubernetes audit logs, Falco JSON output, image digests, service account token exposure checks, and incident ticket references to each detection record.